

Exception Handling in java

Name - S.Trivikram

Roll number - A25126552251

objective, task description, step-wise execution, algorithm, UML diagram, program code, code explanation, expected output, and result.

Java Exception Handling:

ArrayIndexOutOfBoundsException

Program: StudentMarks

1. What is ArrayIndexOutOfBoundsException?

ArrayIndexOutOfBoundsException occurs when a program tries to access an array using an index outside the valid range. In this program, the marks array has three elements, so valid indexes are 0, 1, and 2. Accessing marks[5] causes the exception.

2. Objective

To understand and demonstrate how Java handles ArrayIndexOutOfBoundsException using the try-catch mechanism in a real-world student marks system.

3. Task Description

Create a Java program for a student marks system that performs an operation capable of causing ArrayIndexOutOfBoundsException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required student marks system data or object is created.
3. The operation that may cause ArrayIndexOutOfBoundsException is placed inside the try block.
4. Java detects the invalid condition and throws ArrayIndexOutOfBoundsException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.

Program: StudentMarks

```
int[] marks = {80, 75, 90};
```

```
        try {
            System.out.println("Student Mark = " + marks[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Invalid mark position.");
        }
    }
}
```

8. Code Explanation

Class declaration — Defines the class containing the student marks system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

int[] marks — Creates an integer array containing student marks; marks[5] is outside the valid indexes.

catch (ArrayIndexOutOfBoundsException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Invalid mark position.

10. Result

Thus, the Java program successfully demonstrates `ArrayIndexOutOfBoundsException` and handles the exception using try-catch.

Class declaration — Defines the class containing the student marks system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

int[] marks — Creates an integer array containing student marks; marks[5] is outside the valid indexes.

catch (ArrayIndexOutOfBoundsException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

NullPointerException

Program: Customer

1. What is NullPointerException?

NullPointerException occurs when a program attempts to use an object reference that contains null. Here, customerName is null, so calling length() on it causes the exception.

2. Objective

To understand and demonstrate how Java handles NullPointerException using the try-catch mechanism in a real-world customer details system.

3. Task Description

Create a Java program for a customer details system that performs an operation capable of causing NullPointerException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required customer details system data or object is created.
3. The operation that may cause NullPointerException is placed inside the try block.
4. Java detects the invalid condition and throws NullPointerException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If NullPointerException occurs, transfer control to the catch block.

Program: Customer

```

        System.out.println("Customer Name Length = "
            + customerName.length());
    } catch (NullPointerException e) {
        System.out.println("Error: Customer name is not available.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the customer details system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

customerName = null — Creates a null reference, so calling length() causes the exception.

catch (NullPointerException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Customer name is not available.

10. Result

Thus, the Java program successfully demonstrates NullPointerException and handles the exception using try-catch.

Class declaration — Defines the class containing the customer details system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

customerName = null — Creates a null reference, so calling length() causes the exception.

catch (NullPointerException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

NumberFormatException

Program: BankAccount

1. What is NumberFormatException?

NumberFormatException occurs when a string that does not contain a valid numeric value is converted into a number. The value "ABC" cannot be converted to an integer, so the exception is caught.

2. Objective

To understand and demonstrate how Java handles NumberFormatException using the try-catch mechanism in a real-world bank account system.

3. Task Description

Create a Java program for a bank account system that performs an operation capable of causing NumberFormatException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required bank account system data or object is created.
3. The operation that may cause NumberFormatException is placed inside the try block.
4. Java detects the invalid condition and throws NumberFormatException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If NumberFormatException occurs, transfer control to the catch block.

Program: BankAccount

```

        int amount = Integer.parseInt(balance);
        System.out.println("Balance = " + amount);
    } catch (NumberFormatException e) {
        System.out.println("Error: Invalid balance format.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the bank account system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

balance = "ABC" — Stores a non-numeric string that cannot be converted to an integer.

catch (NumberFormatException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Invalid balance format.

10. Result

Thus, the Java program successfully demonstrates `NumberFormatException` and handles the exception using `try-catch`.

Class declaration — Defines the class containing the bank account system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

balance = "ABC" — Stores a non-numeric string that cannot be converted to an integer.

catch (NumberFormatException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

StringIndexOutOfBoundsException

Program: MobileNumber

1. What is StringIndexOutOfBoundsException?

StringIndexOutOfBoundsException occurs when an invalid position is accessed in a string. The mobile string has indexes 0 through 9, so charAt(15) is invalid.

2. Objective

To understand and demonstrate how Java handles StringIndexOutOfBoundsException using the try-catch mechanism in a real-world mobile number processing.

3. Task Description

Create a Java program for a mobile number processing that performs an operation capable of causing StringIndexOutOfBoundsException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required mobile number processing data or object is created.
3. The operation that may cause StringIndexOutOfBoundsException is placed inside the try block.
4. Java detects the invalid condition and throws StringIndexOutOfBoundsException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If StringIndexOutOfBoundsException occurs, transfer control to the catch block.

Program: MobileNumber

```

        System.out.println("Digit = " + mobile.charAt(15));
    } catch (StringIndexOutOfBoundsException e) {
        System.out.println("Error: Invalid mobile number position.");
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the mobile number processing program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

mobile.charAt(15) — Attempts to access a position outside the string's valid range.

catch (StringIndexOutOfBoundsException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Invalid mobile number position.

10. Result

Thus, the Java program successfully demonstrates `StringIndexOutOfBoundsException` and handles the exception using try-catch.

Class declaration — Defines the class containing the mobile number processing program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

mobile.charAt(15) — Attempts to access a position outside the string's valid range.

catch (StringIndexOutOfBoundsException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

ClassCastException

Program: Employee

1. What is ClassCastException?

ClassCastException occurs when an object is cast to an incompatible class type. The object stored in emp is actually a Developer, so casting it to Manager is invalid.

2. Objective

To understand and demonstrate how Java handles ClassCastException using the try-catch mechanism in a real-world employee management system.

3. Task Description

Create a Java program for a employee management system that performs an operation capable of causing ClassCastException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required employee management system data or object is created.
3. The operation that may cause ClassCastException is placed inside the try block.
4. Java detects the invalid condition and throws ClassCastException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If ClassCastException occurs, transfer control to the catch block.

Program: Employee

```

class EmployeeSystem {
    public static void main(String[] args) {
        Employee emp = new Developer();
        try {
            Manager manager = (Manager) emp;
            System.out.println("Employee is a Manager.");
        } catch (ClassCastException e) {
            System.out.println("Error: Employee is not a Manager.");
        }
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the employee management system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

Manager manager = (Manager) emp — Attempts to cast a Developer object to Manager.

catch (ClassCastException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Employee is not a Manager.

10. Result

Thus, the Java program successfully demonstrates ClassCastException and handles the exception using try-catch.

Class declaration — Defines the class containing the employee management system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

Manager manager = (Manager) emp — Attempts to cast a Developer object to Manager.

catch (ClassCastException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

IllegalArgumentException

Program: DrivingLicense

1. What is IllegalArgumentException?

IllegalArgumentException occurs when a method receives an argument that is not valid for the required operation. In this program, age 16 is below the required age of 18, so the exception is explicitly thrown.

2. Objective

To understand and demonstrate how Java handles IllegalArgumentException using the try-catch mechanism in a real-world driving license validation.

3. Task Description

Create a Java program for a driving license validation that performs an operation capable of causing IllegalArgumentException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required driving license validation data or object is created.
3. The operation that may cause IllegalArgumentException is placed inside the try block.
4. Java detects the invalid condition and throws IllegalArgumentException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IllegalArgumentException occurs, transfer control to the catch block.

Program: DrivingLicense

```
throw new IllegalArgumentException(
```

```

        "Age must be 18 or above.");
    }
    System.out.println("Eligible for driving license.");
}

public static void main(String[] args) {
    try {
        checkAge(16);
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}

```

8. Code Explanation

Class declaration — Defines the class containing the driving license validation program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

checkAge(16) — Passes an age below the required minimum and explicitly throws the exception.

catch (IllegalArgumentException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Age must be 18 or above.

10. Result

Thus, the Java program successfully demonstrates `IllegalArgumentException` and handles the exception using try-catch.

Class declaration — Defines the class containing the driving license validation program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

checkAge(16) — Passes an age below the required minimum and explicitly throws the exception.

catch (IllegalArgumentException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

FileNotFoundException

Program: BankStatement

1. What is FileNotFoundException?

FileNotFoundException occurs when a program attempts to open a file that cannot be found or accessed. The program tries to open bankstatement.txt and handles the exception if the file is unavailable.

2. Objective

To understand and demonstrate how Java handles FileNotFoundException using the try-catch mechanism in a real-world bank statement system.

3. Task Description

Create a Java program for a bank statement system that performs an operation capable of causing FileNotFoundException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required bank statement system data or object is created.
3. The operation that may cause FileNotFoundException is placed inside the try block.
4. Java detects the invalid condition and throws FileNotFoundException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If FileNotFoundException occurs, transfer control to the catch block.

Program: BankStatement

```

class BankStatement {
    public static void main(String[] args) {
        try {
            File file = new File("bankstatement.txt");
            Scanner sc = new Scanner(file);
            System.out.println("Bank statement opened successfully.");
            sc.close();
        } catch (FileNotFoundException e) {
            System.out.println("Error: Bank statement file not found.");
        }
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the bank statement system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

new Scanner(file) — Attempts to open the bank statement file.

catch (FileNotFoundException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Bank statement file not found.

10. Result

Thus, the Java program successfully demonstrates `FileNotFoundException` and handles the exception using `try-catch`.

Class declaration — Defines the class containing the bank statement system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

new Scanner(file) — Attempts to open the bank statement file.

catch (FileNotFoundException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

IOException

Program: EmployeeFile

1. What is IOException?

IOException represents an input/output problem while working with files or other streams. The FileReader operation is placed inside try, and any IOException is handled by catch.

2. Objective

To understand and demonstrate how Java handles IOException using the try-catch mechanism in a real-world employee file system.

3. Task Description

Create a Java program for a employee file system that performs an operation capable of causing IOException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required employee file system data or object is created.
3. The operation that may cause IOException is placed inside the try block.
4. Java detects the invalid condition and throws IOException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IOException occurs, transfer control to the catch block.

Program: EmployeeFile

```
class EmployeeFile {
```

```

public static void main(String[] args) {
    try {
        FileReader file = new FileReader("employee.txt");
        int data = file.read();
        System.out.println("Employee data read successfully.");
        file.close();
    } catch (IOException e) {
        System.out.println("Error: Unable to read employee file.");
    }
}
}

```

8. Code Explanation

Class declaration — Defines the class containing the employee file system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

FileReader/read() — Performs file input that may produce an I/O exception.

catch (IOException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Unable to read employee file.

10. Result

Thus, the Java program successfully demonstrates IOException and handles the exception using try-catch.

Class declaration — Defines the class containing the employee file system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

FileReader/read() — Performs file input that may produce an I/O exception.

catch (IOException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

SecurityException

Program: CompanySecurity

1. What is SecurityException?

SecurityException indicates that an operation is not permitted because of a security restriction. This program deliberately throws SecurityException to demonstrate how it can be caught and handled.

2. Objective

To understand and demonstrate how Java handles SecurityException using the try-catch mechanism in a real-world company security system.

3. Task Description

Create a Java program for a company security system that performs an operation capable of causing SecurityException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required company security system data or object is created.
3. The operation that may cause SecurityException is placed inside the try block.
4. Java detects the invalid condition and throws SecurityException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If SecurityException occurs, transfer control to the catch block.

Program: CompanySecurity

```
throw new SecurityException()
```

```
        "Access denied to company resource.");
    } catch (SecurityException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

8. Code Explanation

Class declaration — Defines the class containing the company security system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

throw new SecurityException — Deliberately generates the exception to demonstrate handling.

catch (SecurityException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Access denied to company resource.

10. Result

Thus, the Java program successfully demonstrates SecurityException and handles the exception using try-catch.

Class declaration — Defines the class containing the company security system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

throw new SecurityException — Deliberately generates the exception to demonstrate handling.

catch (SecurityException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

Java Exception Handling:

IllegalStateException

Program: OnlineOrder

1. What is IllegalStateException?

IllegalStateException occurs when a method is called at an inappropriate time because the object's or application's current state does not allow the operation. Here, an order is cancelled before it has been placed.

2. Objective

To understand and demonstrate how Java handles IllegalStateException using the try-catch mechanism in a real-world online shopping order system.

3. Task Description

Create a Java program for a online shopping order system that performs an operation capable of causing IllegalStateException. Catch the resulting exception and display a meaningful message instead of allowing the program to terminate abnormally.

4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required online shopping order system data or object is created.
3. The operation that may cause IllegalStateException is placed inside the try block.
4. Java detects the invalid condition and throws IllegalStateException.
5. The catch block catches the exception.
6. A meaningful error message is printed.
7. The program completes safely without an unhandled exception.

5. Algorithm

1. Start.
2. Create the required real-world data or object.
3. Enter the try block.
4. Perform the operation that may generate the exception.
5. If IllegalStateException occurs, transfer control to the catch block.

Program: OnlineOrder

```
static void cancelOrder () {
```

```

        if (!orderPlaced) {
            throw new IllegalStateException(
                "Order cannot be cancelled before placing.");
        }
        System.out.println("Order cancelled successfully.");
    }

    public static void main(String[] args) {
        try {
            cancelOrder();
        } catch (IllegalStateException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

8. Code Explanation

Class declaration — Defines the class containing the online shopping order system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

cancelOrder() — Attempts to cancel an order before it has been placed.

catch (IllegalStateException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

Error: Order cannot be cancelled before placing.

10. Result

Thus, the Java program successfully demonstrates `IllegalStateException` and handles the exception using try-catch.

Class declaration — Defines the class containing the online shopping order system program.

main() — The main method is the starting point of execution.

try — Contains code that may generate an exception.

cancelOrder() — Attempts to cancel an order before it has been placed.

catch (IllegalStateException) — Catches the exception generated by the invalid operation.

System.out.println(...) — Prints a user-friendly error message.

All 10 Exceptions

No.	Exception	Application
1	ArrayIndexOutOfBoundsException	Student Marks
2	NullPointerException	Customer Details
3	NumberFormatException	Bank Account
4	StringIndexOutOfBoundsException	Mobile Number Processing
5	ClassCastException	Employee Management
6	IllegalArgumentException	Driving License Validation
7	FileNotFoundException	Bank Statement
8	IOException	Employee File
9	SecurityException	Company Security
10	IllegalStateException	Online Shopping Order